UNIVERSITY OF SCIENCE AND TECHNOLOGY OF HANOI
Department of Information and Communication Technology

WEEKLY INTERNSHIP REPORT

Week Report - Detection Engineering and Validation Phase

Design and Implementation of an IDS/IPS-Based SOC Platform for Detecting and Preventing Broken Access Control Attacks in REST APIs

By

Bui Tat Dat

Cyber Security

Main target:	crAPI vulnerable API laboratory
Monitoring platform:	Wazuh Manager, Indexer, and Dashboard
Detection focus:	JWT bypass, Mass Assignment, SSRF, Injection, BOLA/IDOR, BFLA
Current status:	System and Wazuh agent setup completed; pentesting, rule tuning, attack-pattern log collection, and detection evaluation are in progress

Supervisor: Tran Dai Duong
Organization: Company / Internship Organization

Hanoi, 2026

Abstract

This weekly report documents the transition from the previous infrastructure implementation phase to the active detection-engineering phase of a Docker-based SOC laboratory. The project uses crAPI as the intentionally vulnerable API target and Wazuh as the SIEM/XDR layer for log collection, rule matching, indexing, and dashboard-based analysis.

At the time of this report, the system setup and Wazuh agent deployment are completed. The project is currently focused on four ongoing activities: controlled pentesting against crAPI, custom Wazuh rule refinement, attack-pattern log collection, and detection evaluation. Therefore, this report should not describe pentesting, rule writing, log collection, or detection evaluation as fully completed. Instead, those activities are documented as work in progress with partial evidence where available.

During this week, an initial Wazuh rule set was prepared to detect API abuse patterns related to Broken Object Level Authorization (BOLA/IDOR), Broken Function Level Authorization (BFLA), JWT authentication bypass heuristics, Mass Assignment, Server-Side Request Forgery (SSRF), SQL Injection, NoSQL Injection, sensitive path probing, and authentication endpoint abuse. The rules use Wazuh web access log decoding, HTTP status-code grouping, URL pattern matching, and correlation logic such as repeated events from the same source IP within a defined timeframe.

Controlled testing was started using `curl` requests against the local crAPI endpoint `http://10.10.1.1:8888`. The strongest observed validation evidence is the Wazuh Dashboard Discover view using the query `rule.level > 9`, which returned 11 high-severity hits in the last 24 hours. The visible events show repeated `401` responses for `/identity/api/v2/vehicle/<UUID>/location`, consistent with the BOLA/IDOR vehicle-location rule category. Other attack requests were prepared for SSRF, SQLi/NoSQLi, Mass Assignment, JWT `alg:none`, and BFLA probing, but not all of them are confirmed as matched alerts in the available screenshot.

The report therefore separates implemented infrastructure, ongoing testing work, and confirmed alert evidence. It does not claim successful exploitation of crAPI or final detection accuracy. Instead, it evaluates whether the SOC platform can collect relevant web logs, apply custom Wazuh rules, and generate high-severity security alerts for controlled suspicious API behavior. The main limitation is that some detections rely on indicators appearing in the URL or query string. For POST bodies and Authorization headers, additional application logging, WAF logging, or reverse-proxy log-format changes are required for more accurate detection.

Keywords: SOC, Wazuh, crAPI, API Security, Broken Access Control, BOLA, IDOR, BFLA, SSRF, SQL Injection, JWT, Detection Engineering.

Contents

List of Tables

List of Figures

List of Acronyms

Acronym	Meaning
API	Application Programming Interface
BAC	Broken Access Control
BFLA	Broken Function Level Authorization
BOLA	Broken Object Level Authorization
DFIR	Digital Forensics and Incident Response
DQL	Dashboard Query Language, used in Wazuh Dashboard Discover
IDOR	Insecure Direct Object Reference
IDS	Intrusion Detection System
IPS	Intrusion Prevention System
JWT	JSON Web Token
OWASP	Open Worldwide Application Security Project
SIEM	Security Information and Event Management
SOC	Security Operations Center
SSRF	Server-Side Request Forgery
XDR	Extended Detection and Response

Chapter 1

Weekly Progress Overview

1.1 Purpose of This Weekly Report

The previous weekly report focused on preparing the Docker-based SOC infrastructure: crAPI as the vulnerable API environment, Wazuh as the monitoring layer, and DFIR-IRIS as the future case-management component. The current week moves the project into a more practical phase: writing custom Wazuh rules, generating controlled API attack traffic, and checking whether suspicious behavior appears as alerts in the Wazuh Dashboard.

The purpose of this report is to document what has been implemented and validated this week without overstating the result. The report focuses on detection engineering and validation evidence, not on proving that every vulnerability was successfully exploited. For a SOC/IDS project, a *blocked*, *failed*, or *unauthorized* request can still be valuable because it may represent attacker intent, probing behavior, or an early-stage attack attempt.

1.2 Completed Work This Week

The completed and ongoing tasks are summarized below:

- Completed the SOC lab system setup and Wazuh agent deployment from the previous implementation phase.
- Prepared an initial custom Wazuh rule group for crAPI and bGlobal API security use cases.
- Added initial detection logic for BOLA/IDOR numeric object access and UUID-based vehicle-location access.
- Added correlation rules for repeated object access and repeated vehicle-location enumeration from the same source IP.
- Added draft rules for sensitive path probing, authentication endpoint scanning, and web-brute-force patterns.
- Added heuristic rules for SSRF, SQLi/NoSQLi, BFLA/admin endpoint probing, JWT `alg:none`, and Mass Assignment indicators.
- Started controlled pentesting and API request generation against `10.10.1.1:8888` using `curl`.

- Started attack-pattern log collection and detection evaluation through the Wazuh Dashboard Discover view.
- Confirmed partial high-severity Wazuh alert evidence using `rule.level > 9`, especially for vehicle-location BOLA/IDOR behavior.
- Identified practical logging limitations for POST body and Authorization-header based attacks.

1.3 Current Project Status

Table 1.1: Current weekly status

Task	Status	Notes
System setup	Implemented	crAPI lab, Wazuh stack, and supporting SOC components have been prepared from the implementation phase.
Wazuh agents	Implemented	Agent deployment and log-source mapping are completed; agents are used as the collection layer for Wazuh validation.
Custom Wazuh rules	In progress	Initial rules are written for the target attack categories, but tuning and coverage improvement are still ongoing.
Controlled pentesting	In progress	API attack requests are being executed against the local crAPI endpoint to generate realistic malicious and suspicious traffic.
Attack-pattern log collection	In progress	Logs are being collected from controlled requests; some command loops need to be rerun after fixing the shell PATH issue for `curl`.
BOLA/IDOR alert validation	Partially validated	Dashboard evidence shows high-severity events for UUID vehicle-location access attempts returning `401`.
SSRF, SQLi, JWT, Mass Assignment, BFLA validation	In progress	Rules and test requests exist, but full dashboard-confirmed evidence for each category is still being collected.
Detection evaluation	In progress	Initial rule matching is being checked, but final metrics such as precision, recall, and false-positive rate are not yet finalized.
DFIR-IRIS case automation	Pending	Future step: create cases automatically from selected Wazuh alerts after detection logic is stable.

1.4 Main Result

The main result this week is that the system setup and Wazuh agent layer are complete, while the detection engineering pipeline is now being tested at the rule-matching level. A controlled

suspicious request reaches the crAPI web layer, is written into logs, is decoded by Wazuh as a web access log, is evaluated by custom rules, and appears in Wazuh Dashboard Discover as a high-severity alert. This confirms the practical foundation for the next step: broader test coverage and alert tuning.

Chapter 2

Lab Architecture and Detection Scope

2.1 SOC Lab Architecture Summary

The laboratory architecture keeps the same design direction as the previous phase. The attacker or test client sends requests to the crAPI entry point. The web layer produces access logs. Wazuh agents forward logs to Wazuh Manager. Wazuh Manager applies decoders and rules, sends alerts to Wazuh Indexer, and the SOC analyst reviews alerts through Wazuh Dashboard.

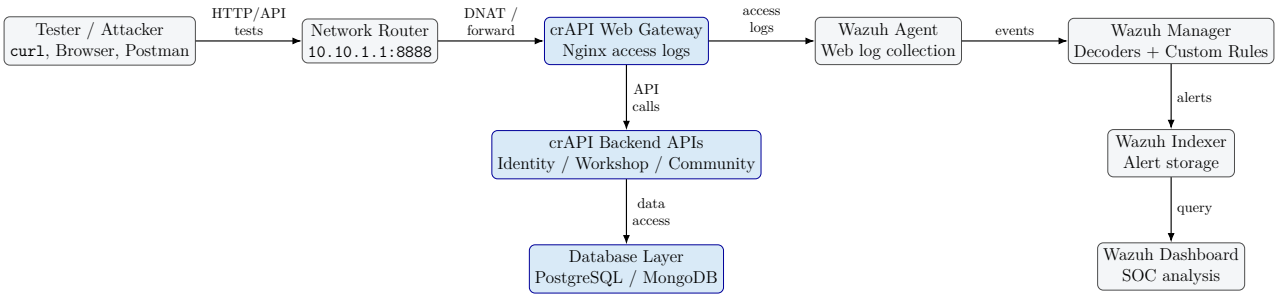

Figure 2.1: Detection workflow for the Wazuh-based crAPI SOC laboratory.

2.2 Detection Scope for This Week

This week focuses on web-access based detections. The rules mainly inspect fields decoded from access logs, especially:

- source IP address, such as `data.srcip`;
- HTTP response code, such as `data.id`;
- HTTP method, such as `data.protocol`;
- request URL, such as `data.url`;
- full original log, such as `full_log`;
- Wazuh metadata, such as `rule.id`, `rule.level`, and `rule.description`.

This scope is suitable for detecting object ID manipulation, suspicious API path probing, and attack indicators visible in the request URL. It is less reliable for attacks carried only in POST bodies or Authorization headers unless those fields are explicitly logged.

2.3 Security Reference Mapping

The custom rules are aligned with OWASP API Security risks and MITRE ATT&CK techniques. OWASP API1:2023 emphasizes that API endpoints commonly expose object identifiers and require object-level authorization checks [3, 4]. OWASP API3:2023 covers object-property authorization issues, including mass assignment-style property manipulation [5]. OWASP API5:2023 covers authorization flaws between administrative and normal functions [6]. OWASP API7:2023 covers SSRF caused by unsafe user-supplied URIs [7]. MITRE ATT&CK mapping is used to classify attack behavior into reconnaissance, public application exploitation, authentication material abuse, and brute force [8, 9, 10, 11].

Table 2.1: Detection category mapping

Category	OWASP API risk	MITRE mapping	Detection idea
BOLA / IDOR	API1:2023	T1190	Detect object identifiers in API paths and repeated access to vehicle-location UUID endpoints.
Mass Assignment	API3:2023	T1190	Detect suspicious object-property fields such as `role`, `isAdmin`, `balance`, and `permission`.
BFLA / Admin probing	API5:2023	T1595 / T1190	Detect access or probing of `/admin`, `/internal`, and `/management` paths.
SSRF	API7:2023	T1190	Detect URLs that attempt to force the server to contact loopback, metadata, or private IP ranges.
SQLi / NoSQLi	API8-style injection behavior	T1190	Detect injection tokens such as `or 1=1`, `union select`, `-`, `$ne`, and `$where`.
JWT bypass heuristic	API2:2023	T1550	Detect `alg:none` or base64 indicators of unsigned JWT abuse when visible in logged request data.
Authentication abuse	API2:2023	T1110 / T1595	Detect repeated failed logins and unknown authentication endpoint scanning.

Chapter 3

Custom Wazuh Rule Engineering

3.1 Rule File Organization

The custom rule file is organized under the group name `bglobal,crapi,api,broken_access_control`. This follows the Wazuh model where XML rules define conditions for interpreting log data and generating alerts [1, 2]. The rule design follows a layered structure:

1. low or medium base rules detect individual suspicious requests;
2. high-severity rules detect confirmed suspicious patterns or important failed attempts;
3. correlation rules use frequency, timeframe, and same-source-IP logic to detect repeated behavior;
4. each rule includes a descriptive group name for filtering and dashboard analysis;
5. selected rules include MITRE ATT&CK technique IDs.

The parent Wazuh web access log rules are used to separate response classes:

- `31108`: commonly used for 2xx/3xx web access events;
- `31101`: commonly used for 4xx web access events;
- `31122`: commonly used for 5xx web access events.

3.2 Rule Severity Model

The rule severity model is designed for SOC triage. Level 8 usually means suspicious behavior that should be reviewed. Level 9 to 10 means the request pattern is strongly attack-like or targets a sensitive API. Level 11 to 13 means the rule is correlated or repeated and should be treated as a stronger incident candidate.

Table 3.1: Severity strategy used in the custom rule set

Level range	Meaning in this project	SOC handling
3 to 6	Informational or low-confidence activity, such as a normal authentication attempt or a single failed login.	Keep for context and correlation; not necessarily incident-worthy alone.
8 to 9	Suspicious path/object access or potentially malicious failed request.	Review during hunting; useful for dashboards and trend analysis.
10	High-confidence attack pattern or important failed attack attempt.	Investigate source IP, endpoint, response code, and related logs.
11 to 13	Repeated or correlated behavior from the same source IP.	Create incident candidate; consider blocking, rate limiting, or IR case creation.

3.3 Rule Summary

Table 3.2: Implemented custom Wazuh rule categories

Rule ID	Level	Category	Trigger idea	Expected SOC meaning
110100	8	BOLA numeric object access	2xx/3xx object ID path	Successful or redirected object access pattern.
110104	8	BOLA numeric object access	4xx object ID path	Unauthorized object access attempt.
110105	9	BOLA numeric object access	5xx object ID path	Possible malformed request or server-side error caused by object access.
110101	13	BOLA correlation	10 object access events in 300 seconds from same source IP	Possible object enumeration.
110120	8	UUID vehicle location BOLA	Vehicle UUID location endpoint with 2xx/3xx	Possible access to vehicle location object.

Rule ID	Level	Category	Trigger idea	Expected SOC meaning
110121	10	Failed vehicle location BOLA	Vehicle location endpoint with 401/403/404	High-confidence failed BOLA/IDOR attempt.
110122	13	UUID BOLA correlation	5 vehicle location events in 120 seconds from same source IP	Possible UUID-based BOLA enumeration.
110102-110107	8-9	Sensitive path probing	`/.env`, `/.git`, `/swagger`, `/api-docs`, `/phpmyadmin`	Reconnaissance against exposed files or debug surfaces.
110103	12	Sensitive path correlation	10 sensitive path probes in 300 seconds	Repeated scanning from one source.
110200-110205	3-12	Authentication abuse	failed login, auth endpoint scanning, repeated attempts	Brute force or auth surface discovery.
110300-110301	10	SSRF	Contact Mechanic plus private IP	Attempt to make backend contact internal resources.
110310-110311	10	SQLi/NoSQLi	URL contains injection tokens	Web attack attempt against API query parameters.
110400-110402	8-12	BFLA/admin probing	`/admin`, `/internal`, `/management`	Attempt to access privileged function.
110500-110501	12	JWT bypass heuristic	`alg:none` or base64 JWT header indicator	Possible unsigned-token authentication bypass attempt.
110510-110511	10	Mass Assignment	sensitive user-property fields in URL/query	Attempt to modify unauthorized object properties.

3.4 Example: BOLA Vehicle Location Rule

The BOLA vehicle location rule is one of the most important rules because the screenshot evidence shows matching activity for the `/identity/api/v2/vehicle/<UUID>/location` pattern. The single-event rule is level 10 for failed access, while the correlation rule is level 13 for

repeated UUID-based access from the same source IP.

Listing 3.1: Selected BOLA/IDOR vehicle-location rules

```
<rule id="110121" level="10">
  <if_sid>31101</if_sid>
  <id>^(401|403|404)$</id>
  <url type="pcre2">(?i)/identity/api/v[0-9]+/vehicle/.*/location/?</url>
  <options>no_full_log</options>
  <description>bGlobal: Possible BOLA/IDOR - Failed Vehicle Location Object Access</description>
  <mitre>
    <id>T1190</id>
  </mitre>
  <group>bglobal,crapi,api,broken_access_control,bola_uuid_access,</group>
</rule>

<rule id="110122" level="13" frequency="5" timeframe="120" ignore="300">
  <if_matched_group>bola_uuid_access</if_matched_group>
  <same_srcip />
  <options>no_full_log</options>
  <description>bGlobal: Possible BOLA/IDOR Enumeration - Repeated Vehicle Location Object Access</description
     >
  <mitre>
    <id>T1190</id>
  </mitre>
  <group>bglobal,crapi,api,broken_access_control,bola,correlation,</group>
</rule>
```

3.5 Why Failed Requests Still Matter

A response code such as `401`, `403`, `404`, or `405` does not prove that the attacker succeeded. However, it is still valuable for SOC detection because it can reveal:

- unauthorized attempts against object-specific endpoints;
- endpoint enumeration before exploitation;
- repeated discovery of admin or internal functions;
- attack payload testing, even when the application rejects the request;
- suspicious behavior that can be correlated before a successful compromise.

Therefore, the rules are designed to detect both successful-looking responses and failed attack attempts. This is appropriate for IDS/SIEM monitoring because early detection is often more useful than waiting for confirmed exploitation.

Chapter 4

Controlled Pentesting and Validation

4.1 Testing Methodology

The testing was performed in a controlled local lab. The target was the crAPI endpoint exposed through `10.10.1.1:8888`. The requests were generated manually using `curl`. The objective was not to attack a real production system. The objective was to generate representative API abuse patterns and verify whether Wazuh could detect them from logs.

The validation workflow was:

1. select an attack category such as BOLA, SSRF, or SQLi;
2. craft a request that places the indicator in the URL/query string when possible;
3. send the request to the crAPI lab endpoint;
4. confirm the HTTP response code;
5. check Wazuh Dashboard Discover with a filter such as `rule.level > 9`;
6. review `rule.id`, `rule.description`, `data.url`, `data.id`, and `full_log`;
7. record whether the test is confirmed, partial, or invalid.

4.2 Executed Test Requests

Listing 4.1: Representative controlled test requests

```
# SSRF attempt using Contact Mechanic endpoint with loopback target
curl -I "http://10.10.1.1:8888/workshop/api/merchant/contact_mechanic?mechanic_api=http
    ://127.0.0.1:22"

# SQL Injection indicator in query string
curl -I "http://10.10.1.1:8888/identity/api/v2/user/search?name=%27%20OR%201%3D1%20--"

# Mass Assignment indicator in query string
curl -I "http://10.10.1.1:8888/identity/api/v2/user/profile?role=admin&balance=999999"

# JWT alg:none style token in query string
curl -I "http://10.10.1.1:8888/identity/api/v2/user/dashboard?token=eyJhbGciOiJub25lIn0.
    eyJ1c2VyIjoiYWRtaW4ifQ."

# UUID-based BOLA / IDOR vehicle-location enumeration
```

```
for i in {1..7}; do
  curl -s -o /dev/null -w "BOLA Vehicle %{http_code}\n" \
  "http://10.10.1.1:8888/identity/api/v2/vehicle/123e4567-e89b-12d3-a456-42661417400$i/
    location"
  sleep 0.2
done
```

4.3 Observed HTTP Responses

Table 4.1: Observed HTTP response behavior from controlled requests

Test category	Endpoint / indicator	Observed code	Interpretation
SSRF HEAD request	Contact Mechanic endpoint with loopback URL indicator	405	Endpoint allows POST/OPTIONS, not HEAD. Request is still useful to test URL-based SSRF rule visibility.
SSRF POST request	JSON body contains loopback URL indicator	JWT required	Application requires authentication. Detection may fail if the suspicious value is only in request body and not logged.
SQL Injection	`name=' OR 1=1 -`	401 or 404	Request reached the API surface but was not authorized or route did not exist. Useful as an attack-intent test.
Mass Assignment	`role=admin&balance=999999`	401	Not authenticated. Useful to test URL/query visibility for sensitive property names.
JWT `alg:none`	unsigned-token style value in query string	404	Request contains JWT bypass indicator, but endpoint response does not prove exploit success.
BOLA / IDOR vehicle UUID	Vehicle-location UUID endpoint	repeated 401	Strongest available validation evidence; Wazuh screenshot shows high-severity alerts for this path.

4.4 Test Execution Issue: Shell PATH

One group of repeated tests did not execute correctly because the shell environment could not locate `curl`. The terminal returned messages such as `curl: command not found` even though `/bin/curl` and `/usr/bin/curl` existed. This is a local shell PATH issue, not a crAPI or Wazuh issue. Those failed loop attempts should not be used as detection evidence.

The fix is to restore a normal PATH or call curl by absolute path:

Listing 4.2: Fixing curl PATH issue during test execution

```
export PATH=/usr/local/sbin:/usr/local/bin:/usr/sbin:/usr/bin:/sbin:/bin

# or call curl directly
/usr/bin/curl -I "http://10.10.1.1:8888/.env"
```

Chapter 5

Wazuh Dashboard Evidence

5.1 Dashboard Query

The Wazuh Dashboard Discover view was filtered using:

```
rule.level > 9
```

The time range was set to the last 24 hours. The result showed 11 hits. The selected fields visible in the dashboard include `agent.ip`, `data.id`, `data.protocol`, `data.srcip`, `data.url`, `full_log`, `rule.level`, `rule.id`, and `rule.description`.

5.2 Evidence Screenshot

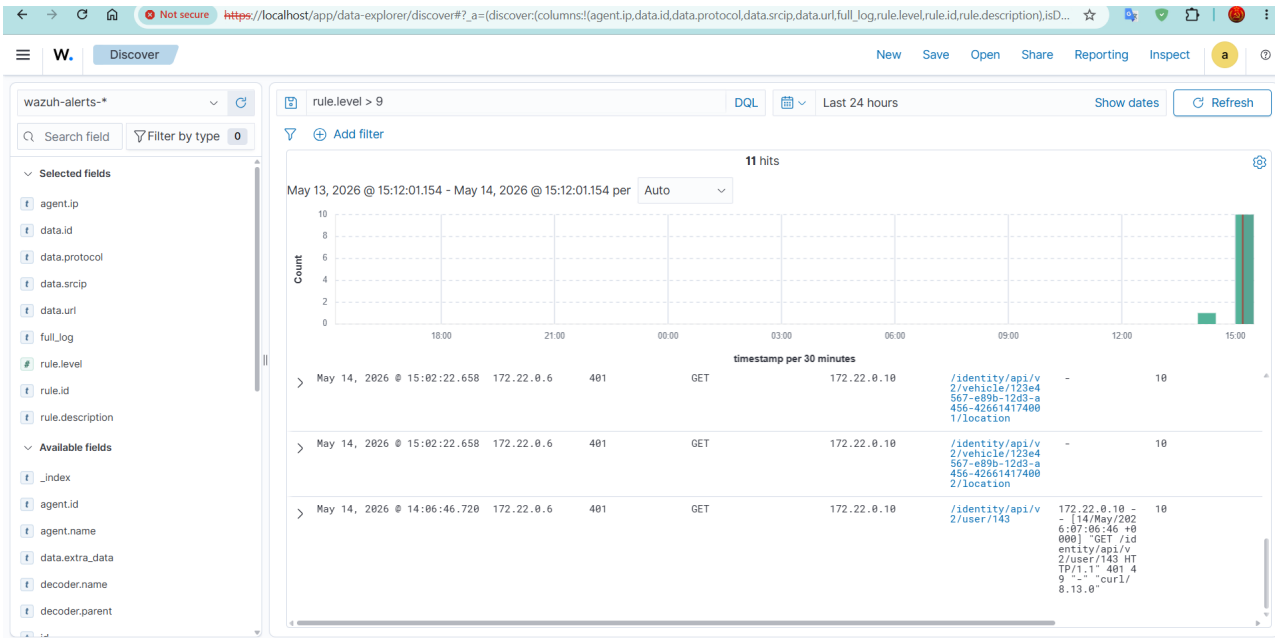

Figure 5.1: Wazuh Dashboard Discover evidence showing high-severity alerts with `rule.level > 9`. The visible rows include `401` GET requests to `/identity/api/v2/vehicle/<UUID>/location`, matching the BOLA/IDOR vehicle-location detection logic.

5.3 Interpretation of the Screenshot

The screenshot provides practical validation that at least part of the custom rule set is working. The important observations are:

- Wazuh indexed events into `wazuh-alerts-*`.
- The Dashboard accepted a severity filter using `rule.level > 9`.
- The search returned 11 hits, showing that high-severity rules fired.
- The visible alerts are related to `GET` requests and `401` responses.
- The visible URL pattern is the vehicle-location UUID endpoint.
- The event fields contain both network-level and rule-level evidence, including `data.srcip`, `data.url`, `rule.level`, and `full_log`.

This evidence is consistent with rule `110121`, which detects failed vehicle-location object access with `401`, `403`, or `404`, and with the broader BOLA/IDOR detection objective. If the repeated access threshold was reached, the event sequence may also support the correlation concept behind rule `110122`. However, the screenshot must be reviewed at row-detail level to confirm the exact `rule.id` for each event.

5.4 What Is Confirmed and What Is Not Confirmed

Table 5.1: Validation confidence from available evidence

Claim	Confidence	Reason
Wazuh Dashboard receives and displays alerts	High	Screenshot shows `wazuh-alerts-*` with 11 hits under `rule.level > 9`.
Custom high-severity rule matching works	High	The visible alert rows show level 10 events matching the vehicle-location BOLA pattern.
BOLA/IDOR failed vehicle-location detection works	High	The visible URL and HTTP status code align with the BOLA/IDOR rule logic.
Repeated BOLA enumeration correlation fired	Medium	Repeated requests were generated, but exact row-level `rule.id` detail is needed to confirm rule `110122`.
SSRF rule matched	Low to medium	SSRF requests were sent, but the available screenshot does not show an SSRF alert row.
SQLi/NoSQLi rule matched	Low to medium	SQLi requests were sent, but the available screenshot does not show a confirmed SQLi rule row.
JWT bypass rule matched	Low to medium	JWT indicator request was sent, but the available screenshot does not confirm rule `110500` or `110501`.
Mass Assignment rule matched	Low to medium	Mass Assignment request was sent, but dashboard proof is not shown in the available evidence.

Chapter 6

Attack Pattern Analysis

6.1 BOLA / IDOR

BOLA/IDOR is the most important attack category in this week's evidence. The attack pattern is based on manipulating an object identifier in the request path. In crAPI, the tested pattern uses a UUID-like vehicle identifier:

`/identity/api/v2/vehicle/<UUID>/location`

The tested requests returned `401`. This means the application did not allow the request without proper authentication. From an exploitation perspective, this is not a successful data access. From a detection perspective, it is still a strong suspicious signal because a tester or attacker is attempting to access object-specific vehicle-location resources.

6.2 BFLA / Admin Endpoint Probing

BFLA focuses on unauthorized access to privileged functions rather than unauthorized access to a specific object. The custom rules detect paths such as:

`/admin`, `/internal`, `/management`

A request such as `/identity/api/v2/admin/dashboard` should be treated as administrative function probing. If repeated from the same source IP, the correlation rule raises severity because this behavior looks like endpoint discovery rather than a single accidental request.

6.3 SSRF

The SSRF test targets the `contact_mechanic` endpoint and attempts to supply a loopback URL:

`mechanic_api=http://127.0.0.1:22`

The HEAD request returned `405 Method Not Allowed`, while the POST request returned a JWT requirement. This is useful because it shows the endpoint behavior, but it also reveals a logging problem: if `mechanic_api` is submitted in the JSON body, default access logs may not contain it. In that case, URL-based Wazuh rules will not match the payload. Accurate SSRF detection should use application logs, WAF logs, or controlled body/header logging.

6.4 SQL Injection and NoSQL Injection

The SQLi/NoSQLi rules search for common injection tokens in the request URL or query string, such as:

- `union select`;
- `or 1=1`;
- `--`;
- `information_schema`;
- `$ne`, `$gt`, `$where`, `$regex`, `$or`, and `$and`.

The current tests generated SQLi-looking query strings, but alert confirmation requires checking whether Wazuh decoded the request URL exactly as expected. URL encoding may also affect matching. A tuning step should normalize or expand patterns for both encoded and decoded variants.

6.5 JWT Authentication Bypass Heuristic

The JWT rules look for `alg:none` or base64 indicators such as `eyJhbGciOiJub25l`. The current test puts the token in the query string to make it visible to the access log. In real attacks, JWTs are normally sent in the `Authorization` header. Therefore, these rules are useful as a heuristic but should not be considered complete JWT protection unless header logging is enabled safely.

6.6 Mass Assignment

The Mass Assignment test uses suspicious user-property fields such as `role=admin` and `balance=999999`. This is an effective test pattern because property names can indicate an attempt to modify server-side object attributes that should not be controlled by a client. Like SSRF and JWT testing, the reliability depends on whether the suspicious field appears in a logged URL/query string or only in a JSON request body.

Chapter 7

SOC Triage Workflow

7.1 Recommended Analyst Workflow

When Wazuh generates an alert for these custom API rules, the analyst should follow a consistent triage process:

1. Identify `rule.id`, `rule.level`, `rule.description`, and `rule.groups`.
2. Confirm the event source: `agent.name`, `agent.ip`, and log location.
3. Review request details: `data.srcip`, `data.protocol`, `data.url`, `data.id`, and `full_log`.
4. Determine whether the event is a single request, repeated sequence, or correlated pattern.
5. Map the alert to OWASP API risk and MITRE technique.
6. Check whether the request was blocked, unauthorized, not found, or successful.
7. Search for related events from the same source IP within 5 to 15 minutes.
8. Decide whether to create an incident case, add the source IP to a watchlist, or tune the rule.

7.2 Example Triage for BOLA Alert

Table 7.1: Example SOC triage worksheet for BOLA/IDOR vehicle-location alert

Triage field	Expected analysis
Alert category	BOLA/IDOR failed vehicle-location object access.
Likely rule	`110121`; check row details to confirm exact `rule.id`.
Source IP	Review `data.srcip`; screenshot shows traffic involving `172.22.0.10`.
Endpoint	`/identity/api/v2/vehicle/<UUID>/location`.
HTTP method and status	`GET` with `401`.
Initial interpretation	Unauthorized object-level access attempt or test request against object-specific endpoint.
Correlation step	Search same `data.srcip` and same path family over the last 5 to 15 minutes.
Escalation condition	Multiple UUIDs, repeated 401/403/404, unusual user-agent, or transition from 4xx to 2xx.
Response recommendation	Keep as alert evidence; optionally add rate-limit/blocking rule after false-positive review.

7.3 Incident Response Integration Plan

For the next project phase, high-severity Wazuh alerts can be forwarded to DFIR-IRIS or another incident-response tool. The recommended case fields are:

- case title: `Possible BOLA/IDOR Enumeration Against crAPI`;
- severity: based on Wazuh `rule.level`;
- source IP: from `data.srcip`;
- affected endpoint: from `data.url`;
- evidence: `full_log`, timestamp, `rule.id`, and screenshot link;
- mapped risk: OWASP API1:2023 and MITRE T1190;
- response tasks: verify source, review request count, block/rate-limit if repeated, and preserve logs.

Chapter 8

Limitations and Tuning Plan

8.1 Current Limitations

The current detection implementation is useful but not complete. The main limitations are:

- **URL visibility dependency:** Several rules only work if the suspicious indicator is present in `data.url`.
- **POST body blind spot:** JSON bodies are not normally stored in default Nginx access logs.
- **Authorization header blind spot:** JWT attacks usually appear in headers, not in the URL.
- **Encoding variation:** SQLi and NoSQLi payloads may be URL-encoded, double-encoded, or split across parameters.
- **No labeled dataset yet:** The project cannot yet calculate precision, recall, false positives, or false negatives.
- **No automatic response yet:** Alerts are visible, but active response and incident-case creation are still future work.

8.2 Recommended Tuning Improvements

1. Add application-level logging for authentication failures, token parsing errors, authorization denials, and business-logic exceptions.
2. Add a safe reverse-proxy log format that captures method, path, query string, status, upstream status, user-agent, request time, and correlation ID.
3. Avoid logging sensitive secrets in full. If headers or body fields are needed, mask tokens and passwords.
4. Add dedicated rules for user-agent anomalies and request-rate anomalies.
5. Add correlation using `same_url`, `same_srcip`, and potentially dynamic fields such as object ID.
6. Create a benign baseline dataset from normal crAPI user behavior.

7. Create a malicious dataset for each attack category with known expected rule IDs.
8. Evaluate true positives, false positives, false negatives, and rule coverage.

8.3 Improved Evidence Collection Commands

The following commands can help make future evidence more complete:

Listing 8.1: Useful validation commands for future report evidence

```
# Check Wazuh agent list
/var/ossec/bin/agent_control -l

# Test a single raw log line against Wazuh rules
/var/ossec/bin/wazuh-logtest

# Search high-severity alerts in Wazuh Dashboard
rule.level > 9

# Recommended fields to add in Discover
agent.name, agent.ip, data.srcip, data.id, data.protocol, data.url, rule.id, rule.level,
     rule.description, full_log
```

Chapter 9

Conclusion and Next Week Plan

9.1 Conclusion

This week demonstrates measurable progress in the SOC/IDS project. The project moved from infrastructure readiness to practical detection engineering. Custom Wazuh rules were written for several API attack categories, including BOLA/IDOR, BFLA, SSRF, SQLi/NoSQLi, JWT bypass heuristics, Mass Assignment, sensitive path probing, and authentication abuse. Controlled requests were generated against the crAPI lab, and the Wazuh Dashboard showed 11 high-severity hits under the filter `rule.level > 9`.

The strongest confirmed evidence is for BOLA/IDOR-style vehicle-location access attempts. The available screenshot shows repeated `401` responses for `/identity/api/v2/vehicle/<UUID>/location`, which is consistent with the failed vehicle-location BOLA rule logic. Other attack categories were configured and tested at the request level, but they require additional dashboard evidence or improved logging before they can be claimed as fully validated.

The project should now focus on rule tuning, evidence quality, and dataset-based evaluation. The most important engineering improvement is to add safe application or WAF logging for request bodies and Authorization headers, because several API attacks do not expose their indicators in the URL. This will make SSRF, JWT, and Mass Assignment detection more reliable.

9.2 Next Week Plan

1. Open the row details for each Wazuh alert and record exact `rule.id`, `rule.description`, and `full_log`.
2. Re-run all attack test categories after fixing the shell PATH issue.
3. Build a table of expected rule ID versus observed rule ID.
4. Add safe logging for POST body indicators where appropriate.
5. Add JWT/header visibility through application logs or controlled WAF logs, with token masking.
6. Create a normal traffic baseline and a malicious test dataset.
7. Calculate detection metrics for each category: true positive, false positive, false negative, and rule coverage.

8. Prepare a Wazuh dashboard panel for high-severity API attack categories.
9. Start Wazuh-to-DFIR-IRIS case creation for level 12 and 13 alerts.

Appendix A

Appendix: Rule Validation Checklist

A.1 Expected Rule Coverage Matrix

Table A.1: Expected validation matrix for future testing

Attack scenario	Test indicator	Expected rule ID	Current evidence level
BOLA numeric object access	`/api/v2/user/143`	110100 / 110104 / 110105 / 110101	Partial
BOLA vehicle UUID access	Vehicle-location UUID endpoint	110120 / 110121 / 110122	Strongest evidence
Sensitive path probing	`/.env`, `/.git/config`, `/swagger`	110102 / 110106 / 110107 / 110103	Pending valid rerun
Authentication brute force	repeated failed login POSTs	110200 / 110201 / 110203	Pending valid rerun
Auth endpoint scanning	unknown auth path	110204 / 110205	Partial
SSRF	Contact Mechanic with loopback/private IP	110300 / 110301	Request sent, alert not confirmed
SQLi/NoSQLi	`' OR 1=1 -`, `$ne`, `$where`	110310 / 110311	Request sent, alert not confirmed
BFLA	`/admin`, `/internal`, `/management`	110400 / 110401 / 110402	Pending valid rerun
JWT bypass heuristic	`alg:none`, `eyJhbGciOiJub25l`	110500 / 110501	Request sent, alert not confirmed
Mass Assignment	`role=admin`, `balance=999999`, `permission`	110510 / 110511	Request sent, alert not confirmed

A.2 Recommended Report Evidence Format

For each detection rule, future weekly reports should include the following evidence fields:

- test name;
- exact command or request;
- observed HTTP status code;
- raw access log line;
- Wazuh `rule.id`;
- Wazuh `rule.description`;
- Wazuh alert screenshot;
- analyst interpretation;
- tuning decision.

References

[1] Wazuh Documentation, "Rules Syntax - Ruleset XML syntax." URL: https://documentation.wazuh.com/current/user-manual/ruleset/ruleset-xml-syntax/rules.html

[2] Wazuh Documentation, "Custom rules." URL: https://documentation.wazuh.com/current/user-manual/ruleset/rules/custom.html

[3] OWASP API Security Project, "OWASP Top 10 API Security Risks - 2023." URL: https://owasp.org/API-Security/editions/2023/en/0x11-t10/

[4] OWASP API Security Project, "API1:2023 Broken Object Level Authorization." URL: https://owasp.org/API-Security/editions/2023/en/0xa1-broken-object-level-authorization/

[5] OWASP API Security Project, "API3:2023 Broken Object Property Level Authorization." URL: https://owasp.org/API-Security/editions/2023/en/0xa3-broken-object-property-level-authorization/

[6] OWASP API Security Project, "API5:2023 Broken Function Level Authorization." URL: https://owasp.org/API-Security/editions/2023/en/0xa5-broken-function-level-authorization/

[7] OWASP API Security Project, "API7:2023 Server Side Request Forgery." URL: https://owasp.org/API-Security/editions/2023/en/0xa7-server-side-request-forgery/

[8] MITRE ATT&CK, "T1190: Exploit Public-Facing Application." URL: https://attack.mitre.org/techniques/T1190/

[9] MITRE ATT&CK, "T1595: Active Scanning." URL: https://attack.mitre.org/techniques/T1595/

[10] MITRE ATT&CK, "T1550: Use Alternate Authentication Material." URL: https://attack.mitre.org/techniques/T1550/

[11] MITRE ATT&CK, "T1110: Brute Force." URL: https://attack.mitre.org/techniques/T1110/